```
import { useState } from "react";

const WARNING = `⚠️  HONEST CONSTRAINT — READ FIRST
Uber's public Rides API (POST /v1.2/requests) was closed to new apps in 2019.
You cannot programmatically book a consumer ride without an enterprise Uber for B

What this agent does instead (and it's still very useful):
  1. Polls Gmail for Sameer's email → parses 2:35 or 4 PM
  2. Builds an Uber Universal Deep Link (m.uber.com/ul/) pre-filled with JP Steve
  3. Opens that link on your Mac AND texts it to Sameer via Twilio
  4. Sameer taps the SMS link on his iPhone → Uber app opens, ride pre-filled → o
  5. macOS notification confirms the whole chain fired

Net result: 1 email from Sameer → 1 tap on his phone to confirm. That's the reali

const FILES = {
  "SKILL.md": {
    lang: "markdown",
    code: `---
name: uber-ride-agent
description: >
  Polls Gmail for ride-request emails from Sameer, parses pickup time (2:35 PM or
  opens a pre-filled Uber deep link, and sends an SMS + macOS notification.
  Trigger phrases: 'uber ride', 'sameer ride', 'school pickup', 'ride agent'.
  Runs as a macOS launchd service on an M4 Mac.
---

# Uber Ride Agent

Automates Sameer's school pickup request from JP Stevens HS, Edison NJ 08820.
Gmail → Parse Time → Uber Deep Link → SMS (Twilio) → macOS Notification.

## Constraint
Uber's consumer Rides API is closed to new developers (2019).
This skill uses Uber Universal Deep Links + Twilio SMS. Sameer taps Confirm once.

## Workflow
1. launchd fires agent.py every 120 seconds
2. Gmail API queries: from:<sameer_email> subject:uber is:unread
3. email_parser.py extracts 2:35 PM or 4:00 PM from body
4. uber_client.py builds m.uber.com/ul/ deep link (JP Stevens → Home coords)
5. \`open\` launches the URL in default macOS browser
6. notifier.py fires osascript macOS notification
7. Twilio SMS sent to Sameer's phone with the deep link
```

```
8. Email marked as read

## Email Format (Sameer sends)
Subject: uber 2:35   (or: uber 4)
Body:    2:35         (or: 4pm)

## Scripts
- scripts/agent.py          — entry point, orchestrates the flow
- scripts/gmail_poller.py   — Gmail API OAuth2 polling
- scripts/email_parser.py   — time extraction (2:35 or 4pm)
- scripts/uber_client.py    — Uber deep link builder
- scripts/notifier.py       — macOS osascript + Twilio SMS

## References
- references/email-format.md   — what Sameer should write
- references/config-setup.md   — one-time credential setup

## Rules
- Never book a ride more than 90 minutes in the future
- Never process emails older than 6 hours
- Mark email as read immediately after processing (even on error)
- Log everything to ~/Library/Logs/uber-agent.log
`
  },

  "config.yaml": {
    lang: "yaml",
    code: `# ── Uber Ride Agent Config ──────────────────────────────────────
# Copy to: ~/.config/uber-agent/config.yaml  (chmod 600)

gmail:
  authorized_sender: "sameer@gmail.com"        # Sameer's email — ONLY this addre
  credentials_path: "~/.config/uber-agent/gmail_credentials.json"
  token_path:       "~/.config/uber-agent/gmail_token.pkl"

twilio:
  account_sid:  "ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
  auth_token:   "your_twilio_auth_token"
  from_phone:   "+1XXXXXXXXXX"   # Your Twilio number
  sameer_phone: "+1XXXXXXXXXX"   # Sameer's cell

uber:
  # No server_token needed for deep links.
  # Uber for Business server_token goes here if you get enterprise access.
  pickup:
    lat:     40.5312
    lng:    -74.3687
```

```
    address: "JP Stevens High School, 855 Grove Ave, Edison, NJ 08820"
    nickname: "JP Stevens HS"
  dropoff:
    lat:     40.5200      # TODO: replace with real home lat
    lng:    -74.3600      # TODO: replace with real home lng
    address: "Home"       # TODO: replace with real address
    nickname: "Home"

agent:
  poll_interval_sec: 120          # must match launchd StartInterval
  max_email_age_hours: 6          # ignore emails older than this
  max_advance_booking_min: 90     # don't open Uber > 90 min before pickup
  log_path: "~/Library/Logs/uber-agent.log"
`
  },

  "scripts/agent.py": {
    lang: "python",
    code: `#!/usr/bin/env python3
"""
Uber Ride Agent — entry point called by launchd every 120 seconds.
~/Library/Logs/uber-agent.log  for runtime logs.
"""
import logging
import sys
from pathlib import Path
from datetime import datetime, timezone, timedelta

import pytz
import yaml

# local imports (PYTHONPATH set by launchd to this directory)
from gmail_poller import get_gmail_service, poll_for_ride_requests, mark_as_read
from email_parser import parse_pickup_time, time_label
from uber_client   import open_ride_request, build_deep_link
from notifier      import macos_notification, send_sms

EASTERN = pytz.timezone("America/New_York")

# ── Config ──────────────────────────────────────────────────────────────

def load_config() -> dict:
    path = Path.home() / ".config/uber-agent/config.yaml"
    with open(path) as f:
        cfg = yaml.safe_load(f)
    # Expand ~ in nested paths
    cfg["gmail"]["credentials_path"] = str(Path(cfg["gmail"]["credentials_path"])
```

```
    cfg["gmail"]["token_path"]       = str(Path(cfg["gmail"]["token_path"]).expan
    return cfg

# ── Main ──────────────────────────────────────────────────────────────────

def setup_logging(log_path: str):
    Path(log_path).expanduser().parent.mkdir(parents=True, exist_ok=True)
    logging.basicConfig(
        filename=str(Path(log_path).expanduser()),
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

def run():
    cfg = load_config()
    setup_logging(cfg["agent"]["log_path"])
    log = logging.getLogger(__name__)
    log.info("── Agent tick ──────────────────────────────────")

    try:
        service = get_gmail_service(
            token_path=cfg["gmail"]["token_path"],
            credentials_path=cfg["gmail"]["credentials_path"],
        )
    except Exception as e:
        log.error(f"Gmail auth failed: {e}")
        sys.exit(1)

    emails = poll_for_ride_requests(
        service,
        authorized_sender=cfg["gmail"]["authorized_sender"],
        max_age_hours=cfg["agent"]["max_email_age_hours"],
    )

    if not emails:
        log.info("No unread ride requests.")
        return

    # Process the most recent unread email only
    email = emails[0]
    log.info(f"Received: '{email['subject']}' from {email['from']}")

    # Always mark read so we don't reprocess
    mark_as_read(service, email["id"])

    pickup_time = parse_pickup_time(email["body"])
```

```
if pickup_time is None:
    log.warning("Could not parse pickup time. Body: " + email["body"][:120])
    macos_notification(
        title="Uber Agent ⚠️",
        message="Could not parse time from Sameer's email. Check logs.",
    )
    return

now = datetime.now(EASTERN)
advance_min = (pickup_time - now).total_seconds() / 60

if advance_min > cfg["agent"]["max_advance_booking_min"]:
    log.warning(f"Pickup {time_label(pickup_time)} is {advance_min:.0f} min a
    macos_notification(
        title="Uber Agent ⏳",
        message=f"Ride for {time_label(pickup_time)} received but it's {advan
    )
    # Un-mark as read so it gets picked up again closer to the time
    # (re-read won't re-trigger — implement a pending state if needed)
    return

# ── Open Uber deep link ──────────────────────────────────────────────
url = open_ride_request(pickup_time, cfg["uber"])
log.info(f"Uber deep link opened for {time_label(pickup_time)}: {url}")

# ── macOS notification ───────────────────────────────────────────────
macos_notification(
    title="Uber Agent 🚗",
    message=f"Ride request sent for {time_label(pickup_time)}",
    subtitle="Sameer: tap the SMS link → Confirm in Uber",
)

# ── SMS to Sameer ────────────────────────────────────────────────────
sms_body = (
    f"🚗 Your Uber is ready for {time_label(pickup_time)}\\n"
    f"Tap to confirm → {url}"
)
send_sms(
    to=cfg["twilio"]["sameer_phone"],
    from_=cfg["twilio"]["from_phone"],
    body=sms_body,
    account_sid=cfg["twilio"]["account_sid"],
    auth_token=cfg["twilio"]["auth_token"],
)
log.info("SMS sent to Sameer.")
log.info("Done ✓")
```

```
if __name__ == "__main__":
    run()
`
  },

  "scripts/gmail_poller.py": {
    lang: "python",
    code: `"""
Gmail API poller — OAuth2, read/modify scope.
First run opens a browser for consent; token is cached at token_path.
"""
import base64
import pickle
from datetime import datetime, timedelta
from pathlib import Path

import pytz
from google.auth.transport.requests import Request
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

SCOPES = ["https://www.googleapis.com/auth/gmail.modify"]
EASTERN = pytz.timezone("America/New_York")


def get_gmail_service(token_path: str, credentials_path: str):
    """Return an authenticated Gmail service, refreshing/creating creds as needed
    creds = None
    tp = Path(token_path)
    if tp.exists():
        with open(tp, "rb") as f:
            creds = pickle.load(f)
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            flow = InstalledAppFlow.from_client_secrets_file(credentials_path, SC
            creds = flow.run_local_server(port=0)
        with open(tp, "wb") as f:
            pickle.dump(creds, f)
    return build("gmail", "v1", credentials=creds)


def poll_for_ride_requests(service, authorized_sender: str, max_age_hours: int =
    """
    Return list of unread ride-request emails from authorized_sender,
```

```
    sorted newest-first, filtered to max_age_hours.
    """
    query = f"from:{authorized_sender} subject:uber is:unread"
    result = service.users().messages().list(userId="me", q=query, maxResults=10)
    messages = result.get("messages", [])

    cutoff = datetime.now(EASTERN) - timedelta(hours=max_age_hours)
    emails = []

    for msg in messages:
        full = service.users().messages().get(
            userId="me", id=msg["id"], format="full"
        ).execute()

        # Filter by age
        ts_ms = int(full.get("internalDate", 0))
        received = datetime.fromtimestamp(ts_ms / 1000, tz=EASTERN)
        if received < cutoff:
            continue

        headers = {h["name"]: h["value"] for h in full["payload"]["headers"]}
        body = _extract_plain_text(full["payload"])

        emails.append({
            "id":      msg["id"],
            "subject": headers.get("Subject", ""),
            "from":    headers.get("From", ""),
            "body":    body,
            "received": received,
        })

    # Newest first
    emails.sort(key=lambda e: e["received"], reverse=True)
    return emails


def mark_as_read(service, message_id: str):
    service.users().messages().modify(
        userId="me",
        id=message_id,
        body={"removeLabelIds": ["UNREAD"]},
    ).execute()


def _extract_plain_text(payload) -> str:
    """Recursively extract text/plain from MIME payload."""
    if payload.get("mimeType") == "text/plain":
```

```
        data = payload.get("body", {}).get("data", "")
        return base64.urlsafe_b64decode(data + "==").decode("utf-8", errors="igno
    for part in payload.get("parts", []):
        result = _extract_plain_text(part)
        if result:
            return result
    return ""
`
  },

  "scripts/email_parser.py": {
    lang: "python",
    code: `"""
Parse Sameer's ride-request email and return a timezone-aware pickup datetime.

Valid emails (case-insensitive, extra whitespace ignored):
  Body: "2:35"   → 14:35 Eastern today
  Body: "2:35pm" → 14:35 Eastern today
  Body: "4"      → 16:00 Eastern today
  Body: "4pm"    → 16:00 Eastern today
  Body: "4:00"   → 16:00 Eastern today
"""
import re
from datetime import datetime

import pytz

EASTERN = pytz.timezone("America/New_York")

# Canonical allowed pickup slots: (hour, minute) → display label
ALLOWED_SLOTS = {
    (14, 35): "2:35 PM",
    (16,  0): "4:00 PM",
}


def parse_pickup_time(email_body: str) -> datetime | None:
    """
    Returns a timezone-aware datetime for today in Eastern time,
    or None if no recognized slot found.
    """
    text = email_body.strip().lower()
    # Remove all whitespace for easier matching
    compact = re.sub(r"\\s+", "", text)

    hour, minute = None, None
```

```
    # 2:35 variants
    if re.search(r"2:?35", compact):
        hour, minute = 14, 35
    # 4pm / 4:00 / 4:00pm
    elif re.search(r"4(pm|:00|:00pm)?$", compact):
        hour, minute = 16, 0

    if hour is None:
        return None

    now = datetime.now(EASTERN)
    pickup = now.replace(hour=hour, minute=minute, second=0, microsecond=0)

    # Reject if in the past (more than 5 min ago)
    if (pickup - now).total_seconds() < -300:
        return None

    return pickup


def time_label(dt: datetime) -> str:
    """Human-readable Eastern time string: '2:35 PM EST'"""
    return dt.strftime("%-I:%M %p %Z")
`
  },

  "scripts/uber_client.py": {
    lang: "python",
    code: `"""
Uber client — Universal Deep Link approach.

HONEST API STATUS
─────────────────
Uber's consumer Rides API (POST /v1.2/requests) has been closed to
new developer apps since 2019. You cannot book a ride programmatically
without an enterprise Uber for Business contract.

What this module does:
  • Builds an m.uber.com/ul/ Universal Link with pickup/dropoff pre-filled
  • Opens it on macOS via \`open\` (routes to Uber app on iPhone if
    continuity/handoff is on, or shows mobile web with a deep link)
  • The SMS sent by notifier.py puts the same URL directly on Sameer's
    phone — he taps it, Uber app opens pre-filled, one tap to Confirm

If you later obtain Uber for Business API access, swap open_ride_request()
for book_via_rides_api() (stub provided at bottom of this file).
"""
```

```
import subprocess
import urllib.parse
from datetime import datetime


def build_deep_link(pickup_time: datetime, uber_cfg: dict) -> str:
    """
    Uber Universal Link — pre-fills pickup + dropoff.
    https://developer.uber.com/docs/riders/ride-requests/tutorials/deep-links/
    """
    p = uber_cfg["pickup"]
    d = uber_cfg["dropoff"]

    params = {
        "action":                      "setPickup",
        "pickup[latitude]":            p["lat"],
        "pickup[longitude]":           p["lng"],
        "pickup[nickname]":            p.get("nickname", "School"),
        "pickup[formatted_address]":   p["address"],
        "dropoff[latitude]":           d["lat"],
        "dropoff[longitude]":          d["lng"],
        "dropoff[nickname]":           d.get("nickname", "Home"),
        "dropoff[formatted_address]":  d["address"],
    }

    return "https://m.uber.com/ul/?" + urllib.parse.urlencode(params)


def open_ride_request(pickup_time: datetime, uber_cfg: dict) -> str:
    """Open the deep link on macOS and return the URL."""
    url = build_deep_link(pickup_time, uber_cfg)
    subprocess.run(["open", url], check=True)
    return url


# ── Uber for Business stub (enterprise only) ──────────────────────────────
#
# import os, requests
#
# def book_via_rides_api(pickup_time: datetime, uber_cfg: dict) -> dict:
#     """
#     Requires: Uber for Business account + server_token with rides.request scope
#     Step 1: GET /v1.2/estimates/price  → get product_id + fare_id
#     Step 2: POST /v1.2/requests        → creates the ride
#     """
#     token = uber_cfg["server_token"]
#     headers = {
```

```
#         "Authorization": f"Bearer {token}",
#         "Content-Type": "application/json",
#     }
#     p, d = uber_cfg["pickup"], uber_cfg["dropoff"]
#
#     # Fare estimate first (required by API to get fare_id)
#     est = requests.get(
#         "https://api.uber.com/v1.2/estimates/price",
#         params={"start_latitude": p["lat"], "start_longitude": p["lng"],
#                 "end_latitude": d["lat"],   "end_longitude": d["lng"]},
#         headers=headers,
#     ).json()
#
#     product_id = est["prices"][0]["product_id"]   # pick cheapest / uberX
#
#     r = requests.post(
#         "https://api.uber.com/v1.2/requests",
#         json={
#             "product_id":      product_id,
#             "start_latitude":  p["lat"], "start_longitude": p["lng"],
#             "end_latitude":    d["lat"], "end_longitude":   d["lng"],
#         },
#         headers=headers,
#     )
#     r.raise_for_status()
#     return r.json()
`
  },

  "scripts/notifier.py": {
    lang: "python",
    code: `"""
Notifications — macOS osascript + Twilio SMS.
"""
import logging
import subprocess

log = logging.getLogger(__name__)


def macos_notification(title: str, message: str, subtitle: str = ""):
    """
    Send a macOS notification banner via osascript.
    Works headlessly — no GUI session needed for launchd agents
    as long as the plist is in ~/Library/LaunchAgents (user domain).
    """
    parts = [f'display notification "{message}"', f'with title "{title}"']
```

```
    if subtitle:
        parts.append(f'subtitle "{subtitle}"')
    script = " ".join(parts)
    try:
        subprocess.run(["osascript", "-e", script], check=True, capture_output=Tr
    except subprocess.CalledProcessError as e:
        log.warning(f"macOS notification failed: {e.stderr.decode()}")


def send_sms(to: str, from_: str, body: str, account_sid: str, auth_token: str) -
    """Send SMS via Twilio. Returns message SID."""
    try:
        from twilio.rest import Client  # lazy import
        client = Client(account_sid, auth_token)
        msg = client.messages.create(to=to, from_=from_, body=body)
        log.info(f"SMS sent to {to}: {msg.sid}")
        return msg.sid
    except ImportError:
        log.error("twilio package not installed. Run: pip install twilio")
        raise
    except Exception as e:
        log.error(f"SMS failed: {e}")
        raise
`
  },

  "com.sarabillabs.uber-agent.plist": {
    lang: "xml",
    code: `<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.sarabillabs.uber-agent</string>

    <!-- Python interpreter from Homebrew (M4 Mac arm64) -->
    <key>ProgramArguments</key>
    <array>
        <string>/opt/homebrew/bin/python3</string>
        <string>/Users/panchaldineshb/Public/k2a/skills/user/uber-ride-agent/scri
    </array>

    <!-- Poll every 2 minutes -->
    <key>StartInterval</key>
    <integer>120</integer>
```

```
    <!-- Fire immediately on load -->
    <key>RunAtLoad</key>
    <true/>

    <!-- Keep alive on crash -->
    <key>KeepAlive</key>
    <false/>

    <key>StandardOutPath</key>
    <string>/Users/panchaldineshb/Library/Logs/uber-agent.log</string>
    <key>StandardErrorPath</key>
    <string>/Users/panchaldineshb/Library/Logs/uber-agent-error.log</string>

    <key>EnvironmentVariables</key>
    <dict>
        <key>PATH</key>
        <string>/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin</string>
        <!-- Scripts directory on PYTHONPATH so local imports work -->
        <key>PYTHONPATH</key>
        <string>/Users/panchaldineshb/Public/k2a/skills/user/uber-ride-agent/scri
        <key>HOME</key>
        <string>/Users/panchaldineshb</string>
    </dict>

    <key>WorkingDirectory</key>
    <string>/Users/panchaldineshb/Public/k2a/skills/user/uber-ride-agent/scripts<
</dict>
</plist>
`
  },

  "setup.sh": {
    lang: "bash",
    code: `#!/usr/bin/env bash
# One-time setup script for the Uber Ride Agent
# Run as: bash setup.sh
set -euo pipefail

SKILL_DIR="$HOME/Public/k2a/skills/user/uber-ride-agent"
CONFIG_DIR="$HOME/.config/uber-agent"

echo "── Installing Python dependencies ──────────────────────────────────"
pip3 install --upgrade \\
  google-api-python-client \\
  google-auth-httplib2 \\
  google-auth-oauthlib \\
  twilio \\
```

```
  pytz \\
  pyyaml

echo ""
echo "── Creating config directory ──────────────────────────────────────"
mkdir -p "$CONFIG_DIR"
chmod 700 "$CONFIG_DIR"

if [ ! -f "$CONFIG_DIR/config.yaml" ]; then
  cp "$SKILL_DIR/config.yaml" "$CONFIG_DIR/config.yaml"
  echo "  → Copied config.yaml to $CONFIG_DIR"
  echo "  ⚠️  EDIT $CONFIG_DIR/config.yaml before continuing"
  echo "      Set: gmail.authorized_sender, twilio.*, uber.dropoff coords"
fi

echo ""
echo "── Gmail credentials ──────────────────────────────────────────────"
echo "  1. Go to https://console.cloud.google.com"
echo "  2. Create a project → Enable Gmail API"
echo "  3. Credentials → OAuth 2.0 Client ID → Desktop app"
echo "  4. Download JSON → save as $CONFIG_DIR/gmail_credentials.json"
echo "  5. Run once manually to authorize:"
echo "     python3 $SKILL_DIR/scripts/agent.py"
echo "     (browser opens, grant access, token.pkl saved)"

echo ""
echo "── Install launchd agent ──────────────────────────────────────────"
PLIST_SRC="$SKILL_DIR/com.sarabillabs.uber-agent.plist"
PLIST_DST="$HOME/Library/LaunchAgents/com.sarabillabs.uber-agent.plist"

cp "$PLIST_SRC" "$PLIST_DST"
launchctl unload "$PLIST_DST" 2>/dev/null || true
launchctl load   "$PLIST_DST"
echo "  → launchd agent loaded: com.sarabillabs.uber-agent"

echo ""
echo "── Verify ─────────────────────────────────────────────────────────"
launchctl list | grep uber-agent
echo ""
echo "  Logs: tail -f ~/Library/Logs/uber-agent.log"
echo ""
echo "Done ✓"
`
  },

  "references/email-format.md": {
    lang: "markdown",
```

code: `# Email Format — What Sameer Should Send

Gmail Account
Send from: the email address set in config.yaml → gmail.authorized_sender
Send to: dinesh's Gmail account (the one the agent monitors)

Template

Subject: uber 2:35
Body: 2:35

or

Subject: uber 4
Body: 4pm

Accepted Time Values

What Sameer types	Parsed as
2:35	2:35 PM Eastern
2:35pm	2:35 PM Eastern
4	4:00 PM Eastern
4pm	4:00 PM Eastern
4:00	4:00 PM Eastern

What Happens Next

1. Agent picks it up within 2 minutes
2. Uber deep link opens on the Mac
3. **Sameer gets an SMS with a tap-to-confirm link**
4. Sameer taps link → Uber app opens pre-filled with JP Stevens → Home
5. Sameer taps **Confirm** once in the Uber app
6. macOS notification fires on Dinesh's Mac confirming the chain

---

Why Sameer Still Taps Confirm

Uber's public booking API was shut down in 2019.
The deep link pre-fills everything; the one-tap confirm is the

```
minimum interaction required. This is also a safety net so Sameer
can cancel or change the ride if needed.

---

## Troubleshooting

- **No SMS received:** Check Twilio balance and ~/Library/Logs/uber-agent.log
- **Time not parsed:** Make sure the body contains only the time (no extra text)
- **Old email re-processed:** Emails older than 6 hours are ignored; mark as read
`
  }
};

const LANG_COLORS = {
  python: "#3b82f6",
  yaml:   "#10b981",
  bash:   "#f59e0b",
  xml:    "#a78bfa",
  markdown: "#64748b",
};

function syntaxHighlight(code, lang) {
  // Minimal keyword coloring
  if (lang === "python") {
    return code
      .replace(/("""[\s\S]*?""")/g, '<span style="color:#86efac">$1</span>')
      .replace(/(#[^\n]*)/g, '<span style="color:#6b7280">$1</span>')
      .replace(/\b(def|class|import|from|return|if|else|elif|for|in|not|and|or|No
        '<span style="color:#818cf8">$1</span>')
      .replace(/\b(str|int|dict|list|bool|datetime)\b/g,
        '<span style="color:#67e8f9">$1</span>');
  }
  if (lang === "yaml") {
    return code
      .replace(/(#[^\n]*)/g, '<span style="color:#6b7280">$1</span>')
      .replace(/^(\s*)([\w-]+):/gm,
        '$1<span style="color:#67e8f9">$2</span>:');
  }
  if (lang === "bash") {
    return code
      .replace(/(#[^\n]*)/g, '<span style="color:#6b7280">$1</span>')
      .replace(/\b(echo|mkdir|chmod|cp|pip3|launchctl|set)\b/g,
        '<span style="color:#818cf8">$1</span>');
  }
  return code;
}
```

```
export default function UberAgentViewer() {
  const fileNames = Object.keys(FILES);
  const [active, setActive] = useState("SKILL.md");
  const [showWarning, setShowWarning] = useState(true);

  const file = FILES[active];

  return (
    <div style={{
      background: "#0d1117",
      minHeight: "100vh",
      fontFamily: "'JetBrains Mono', 'Fira Code', monospace",
      color: "#e6edf3",
      display: "flex",
      flexDirection: "column",
    }}>
      {/* Header */}
      <div style={{
        background: "#161b22",
        borderBottom: "1px solid #30363d",
        padding: "14px 20px",
        display: "flex",
        alignItems: "center",
        gap: 12,
      }}>
        <span style={{ fontSize: 20 }}>🚗</span>
        <div>
          <div style={{ fontWeight: 700, fontSize: 14, letterSpacing: "0.05em", c
            uber-ride-agent
          </div>
          <div style={{ fontSize: 11, color: "#8b949e", marginTop: 2 }}>
            skills/user/uber-ride-agent · DBS Framework · launchd · Gmail API · T
          </div>
        </div>
        <div style={{ marginLeft: "auto", display: "flex", gap: 8 }}>
          {["#ff5f57","#febc2e","#28c840"].map((c,i) => (
            <div key={i} style={{ width:12, height:12, borderRadius:"50%", backgr
          ))}
        </div>
      </div>

      {/* Warning banner */}
      {showWarning && (
        <div style={{
          background: "#1c1106",
          border: "1px solid #9a6700",
```

```
    borderLeft: "4px solid #d29922",
    margin: "12px 16px 0",
    borderRadius: 6,
    padding: "10px 14px",
    fontSize: 12,
    color: "#d29922",
    lineHeight: 1.6,
    position: "relative",
  }}>
    <button
      onClick={() => setShowWarning(false)}
      style={{
        position:"absolute", top:8, right:10,
        background:"none", border:"none", color:"#d29922",
        cursor:"pointer", fontSize:16, lineHeight:1,
      }}
    >×</button>
    <pre style={{ margin:0, whiteSpace:"pre-wrap", fontFamily:"inherit" }}>
  </div>
)}

<div style={{ display:"flex", flex:1, overflow:"hidden", marginTop:12 }}>
  {/* Sidebar */}
  <div style={{
    width: 220,
    borderRight: "1px solid #30363d",
    overflowY: "auto",
    padding: "8px 0",
    flexShrink: 0,
  }}>
    <div style={{ fontSize:10, color:"#484f58", padding:"4px 16px 8px", tex
      Files
    </div>
    {fileNames.map(name => {
      const lang = FILES[name].lang;
      const dot = LANG_COLORS[lang] || "#8b949e";
      const isActive = active === name;
      const short = name.replace("scripts/","").replace("references/","");
      const prefix = name.startsWith("scripts/") ? "  " : name.startsWith("
      return (
        <button
          key={name}
          onClick={() => setActive(name)}
          style={{
            display:"block", width:"100%", textAlign:"left",
            padding: "6px 16px",
            background: isActive ? "#21262d" : "transparent",
```

```
            border: "none",
            borderLeft: isActive ? "2px solid #58a6ff" : "2px solid transpa
            cursor: "pointer",
            color: isActive ? "#e6edf3" : "#8b949e",
            fontSize: 12,
            fontFamily: "inherit",
            transition: "all 0.1s",
          }}
        >
          <span style={{
            display:"inline-block", width:6, height:6,
            borderRadius:"50%", background:dot,
            marginRight:8, verticalAlign:"middle",
          }} />
          {prefix}{short}
        </button>
      );
    })}

    {/* Flow diagram */}
    <div style={{
      margin: "16px 12px 0",
      padding: "12px",
      background: "#161b22",
      borderRadius: 6,
      border: "1px solid #30363d",
      fontSize: 10,
      color: "#8b949e",
      lineHeight: 1.8,
    }}>
      <div style={{ color:"#58a6ff", marginBottom:4, fontWeight:700 }}>FLOW
      <div>📧 Sameer emails</div>
      <div style={{paddingLeft:8}}>↓ ~2 min</div>
      <div>⚙️ Gmail poll</div>
      <div style={{paddingLeft:8}}>↓</div>
      <div>🔍 Parse time</div>
      <div style={{paddingLeft:8}}>↓</div>
      <div>🔗 Uber deep link</div>
      <div style={{paddingLeft:8}}>↓</div>
      <div>📱 SMS to Sameer</div>
      <div style={{paddingLeft:8}}>↓</div>
      <div>🔔 Mac notification</div>
      <div style={{paddingLeft:8}}>↓</div>
      <div>✅ Sameer taps Confirm</div>
    </div>
  </div>
```

```
{/* Code pane */}
<div style={{ flex:1, overflow:"auto", padding:0 }}>
  {/* Tab bar */}
  <div style={{
    background:"#161b22",
    borderBottom:"1px solid #30363d",
    padding:"6px 16px 0",
    display:"flex",
    alignItems:"center",
    gap:4,
  }}>
    <div style={{
      padding:"4px 12px",
      background:"#0d1117",
      border:"1px solid #30363d",
      borderBottom:"1px solid #0d1117",
      borderRadius:"4px 4px 0 0",
      fontSize:12,
      color:"#e6edf3",
      marginBottom:-1,
    }}>
      <span style={{
        display:"inline-block", width:8, height:8, borderRadius:"50%",
        background: LANG_COLORS[file.lang] || "#8b949e",
        marginRight:6, verticalAlign:"middle",
      }} />
      {active}
    </div>
  </div>

  {/* Code */}
  <pre style={{
    margin:0,
    padding:"20px 24px",
    fontSize:12,
    lineHeight:1.7,
    color:"#e6edf3",
    overflow:"auto",
  }}>
    <code
      dangerouslySetInnerHTML={{
        __html: syntaxHighlight(
          file.code.replace(/</g,"&lt;").replace(/>/g,"&gt;"),
          file.lang
        )
      }}
    />
```

```
            </pre>
          </div>
        </div>

        {/* Footer */}
        <div style={{
          borderTop:"1px solid #21262d",
          padding:"8px 20px",
          display:"flex",
          alignItems:"center",
          gap:16,
          fontSize:11,
          color:"#484f58",
          background:"#161b22",
        }}>
          <span>📂 10 files</span>
          <span>🐍 Python 3.11+</span>
          <span>⚙️  launchd StartInterval=120</span>
          <span>📍 JP Stevens HS, Edison NJ 08820</span>
          <span style={{marginLeft:"auto"}}>k2a · SarabiLabs</span>
        </div>
      </div>
    );
}
```